Education Evaluation Service

Technical Note — Bodhrik Full Stack Assessment

Candidate: G Sai Pavithra | **GitHub Repository:** https://github.com/Pavithra8805/ed-eval-service
Tech Stack: FastAPI · PostgreSQL · Redis · Docker · Pytest

1. Database Schema Design & Normalization

The database follows **Third Normal Form (3NF)** to ensure data consistency and eliminate redundancy. The system consists of four primary entities: **users**, **students**, **sessions**, and **evaluations**.

- **users Table:** Acts as the central identity system storing authentication credentials and account roles (*admin*, *teacher*, *parent*). Enforcing roles at the database level prevents invalid authorization states.
- **students Table:** Modeled separately from users because students are learners, not login accounts. Each student references their parent's account via a *parent_id* foreign key, enabling clear ownership and parental access controls.
- **sessions Table:** Represents 1:1 tutoring sessions between a teacher and a student. Direct foreign keys (*teacher_id*, *student_id*) eliminate unneeded complexity while making schedule queries fast.
- **evaluations Table:** Kept distinct from sessions to allow multiple evaluation attempts per session (e.g. re-grading). Each evaluation tracks its own asynchronous lifecycle: *pending* → *processing* → *completed* / *failed*.
- **UUID Primary Keys:** Using UUIDs instead of auto-incrementing integers prevents ID guessing in public API endpoints and allows reliable, distributed primary key generation across services.

2. Scaling Role-Based Access Control (RBAC)

The current RBAC model implements role checks directly at the endpoint level. To scale this system for additional roles (e.g., *school_admin*) or multi-tenant organizational hierarchies, the architecture can evolve as follows:

- **Organization Hierarchy:** Add an *organisations* table with parent-child relationships (e.g. School → District). Assigning an *org_id* foreign key to users, students, and sessions enables enterprise multi-tenancy.
- **Granular Permissions System:** Decouple roles from endpoints by introducing a permission-based system (e.g. *session:read*, *evaluation:create*). Endpoints verify specific permissions rather than hardcoded roles.
- **Database Row-Level Security (RLS):** Use PostgreSQL RLS policies to enforce organizational data boundaries directly inside the database engine for seamless security across all queries.

3. Production Safety & Engineering Readiness

To prepare this evaluation platform for high-traffic production environments, the following production enhancements are recommended:

- **Automated Migration Workflows:** Run Alembic schema migrations (`alembic upgrade head`) via CI/CD release pipelines or Kubernetes init-containers, disabling automatic runtime table creation in production.

• **Secure Secrets Management:** Load sensitive configurations (database URI, JWT secret key, Redis password) from environment secret managers (AWS Secrets Manager, HashiCorp Vault, or Kubernetes Secrets).

• **Reliable Job Queuing:** Enhance the Redis evaluation worker with reliable queue patterns (e.g. Redis Streams or Celery with RabbitMQ) to support job retries, dead-letter queues, and at-least-once task delivery.

• **Enhanced Security & Observability:** Implement rate limiting, HTTP CORS domain restrictions, short-lived JWT access tokens with refresh tokens, and structured JSON logging for monitoring and alerting.